adapty | Docs

INTEGRATION GUIDE

Display flows in iOS SwiftUI apps

Minimal setup to fetch and present an Adapty flow in a SwiftUI app — three steps and four required callbacks. Works with both Flow Builder and Paywall Builder placements.

SwiftUI | iOS SDK v4 (beta) | Flow Builder + Paywall Builder

PLATFORM	SDK VERSION	DOCUMENT
iOS · SwiftUI	v4.0 (beta)	Integration Guide

This guide shows the minimum code needed to fetch a flow from Adapty and present it in a SwiftUI app. The same three steps work for placements created in the Flow Builder or in the classic Paywall Builder — no dashboard changes required.

CONTENTS

1. Get the flow ›
2. Get the flow configuration ›
3. Present the flow ›
Full example ›

1. Get the flow

Fetch the flow object by placement ID. The flow is the placement's content for the current user — products, A/B test variant, and the view configuration link.

```swift
import Adapty

let flow = try await Adapty.getFlow(placementId: "YOUR_PLACEMENT_ID")
```

Optional parameters:

Parameter	Default	What it affects
`fetchPolicy`	`.reloadRevalidatingCacheData`	Cache strategy. `.returnCacheDataElseLoad` returns cache when present; `.returnCacheDataIfNotExpiredElseLoad(maxAge: )` returns cache only if newer than `maxAge` seconds.
`loadTimeout`	`5s`	Maximum time to wait for a network response before falling back to cached data or throwing.

2. Get the flow configuration

Fetch the visual configuration — the assets and layout the SDK needs to render the flow on screen.

```swift
import AdaptyUI

let flowConfiguration = try await AdaptyUI.getFlowConfiguration(forFlow: flow)
```

Optional parameters:

Parameter	Default	What it affects
`loadTimeout`	`5.0`	Maximum time to wait for the view configuration to load. Minimum 1.0 second.
`products`	`nil`	Pre-loaded `[AdaptyPaywallProduct]` to skip the SDK's internal product fetch. Pass the result of `Adapty.getPaywallProducts(flow:)` if you have already loaded them.
`observerModeResolver`	`nil`	Required only when the SDK is initialized in observer mode. Bridges Adapty purchase events to your existing purchase code.
`tagResolver`	`nil`	Closure that returns string values for custom `{tag}` placeholders defined in the paywall text.
`timerResolver`	`nil`	Closure that returns end dates for personalized countdown timers configured in the paywall.
`systemRequestsHandler`	`nil`	Hook for the SDK's system-level network requests. Advanced — leave `nil` for normal use.

3. Present the flow

Apply the `.flow(...)` modifier to any view and toggle the `isPresented` binding to show the flow. The modifier takes the loaded configuration plus four callbacks that handle outcomes the SDK can't decide for you:

```swift
import AdaptyUI

struct ContentView: View {
    @State private var flowPresented = false
    @State private var flowConfiguration: AdaptyUI.FlowConfiguration?
```

```
    var body: some View {
        Button("Show paywall") { flowPresented = true }
            .flow(
                isPresented: $flowPresented,
                flowConfiguration: flowConfiguration,
                didFailPurchase: { _, error in
                    // Show an error to the user.
                },
                didFinishRestore: { profile in
                    // Check profile.accessLevels and dismiss if access is granted.
                },
                didFailRestore: { error in
                    // Show an error to the user.
                },
                didFailRendering: { error in
                    flowPresented = false
                    // Surface the rendering error.
                }
            )
    }
}
```

Every other parameter has a sensible default. The modifier handles the `.close` action automatically, opens URLs in Safari, and dismisses on successful purchase. Pass an optional parameter only when you need to override that behavior or hook in your own logic.

Parameter	Default	What it affects
`fullScreen`	`true`	When `true`, the flow uses `.fullScreenCover`. Set to `false` to use `.sheet` (swipe-to-dismiss).
`placeholderBuilder`	`AdaptyLoadingPlaceholderView()`	View shown while `flowConfiguration` is `nil` (still loading). Provide your own `View` to match your app's loading style.
`didAppear`	No-op.	Closure called when the flow becomes visible. Use to log an analytics event.

Parameter	Default	What it affects
`didDisappear`	No-op.	Closure called when the flow is dismissed.
`didPerformAction`	`.close` dismisses; `.openURL` opens the URL in Safari; `.custom` is a no-op.	Override to handle a custom action button, or to replace the close or open URL behavior.
`didSelectProduct`	No-op.	Closure called when the user taps a product.
`didStartPurchase`	No-op.	Closure called when the user initiates a purchase. Show a loading indicator if needed.
`didFinishPurchase`	Dismisses the flow on successful purchase.	Override to run your own post-purchase logic before or instead of dismissing.
`didStartRestore`	No-op.	Closure called when the user initiates restore.
`didFailLoadingProducts`	Returns `true` (retry).	Return `false` to stop retrying after a products fetch failure.
`didPartiallyLoadProducts`	No-op.	Closure called with the IDs of products that failed to load.
`didFinishWebPaymentNavigation`	No-op.	Closure called when a web-based payment flow completes.
`didReceiveAnalyticEvent`	No-op.	Closure called for flow-level analytics events. Use it to forward

Parameter	Default	What it affects
		events to Amplitude, Mixpanel, etc.
`showAlertItem` / `showAlertBuilder`	No alert.	Bind an `Identifiable` item and a builder closure to present a SwiftUI `Alert` on top of the flow.

Full example

SWIFT

```swift
import Adapty
import AdaptyUI
import SwiftUI

struct PaywallView: View {
    @State private var flowPresented = false
    @State private var flowConfiguration: AdaptyUI.FlowConfiguration?

    var body: some View {
        Button("Show paywall") { flowPresented = true }
            .flow(
                isPresented: $flowPresented,
                flowConfiguration: flowConfiguration,
                didFailPurchase: { _, error in
                    print("Purchase failed: \(error)")
                },
                didFinishRestore: { profile in
                    print("Restore finished — check access levels and dismiss if granted.")
                },
                didFailRestore: { error in
                    print("Restore failed: \(error)")
                },
                didFailRendering: { error in
                    flowPresented = false
                    print("Rendering failed: \(error)")
                }
            )
            .task {
                guard flowConfiguration == nil else { return }
                do {
                    let flow = try await Adapty.getFlow(placementId: "YOUR_PLACEMENT_ID")
                    flowConfiguration = try await AdaptyUI.getFlowConfiguration(forFlow: flow)
                } catch {
                    print("Failed to load flow: \(error)")
                }
            }
    }
}
```